Content-Addressing Formal Mathematics

Marcello Paris
marcello.paris@gmail.com

March 2026

Abstract

We explore content addressing for formal mathematics, introducing a stable identifier determined by mathematical content rather than human chosen names. By computing a cryptographic hash of a canonicalized type expression, we can globally address mathematical facts. We hope this approach helps enable a new paradigm for decentralized knowledge sharing, contribution, and discovery. Building on these addresses, we constructed a decentralized registry where the Lean verifier itself serves as the consensus mechanism. We share a working prototype for Lean 4, evaluate it on standard Mathlib modules, and openly discuss our design space, current limitations, and connections to both content addressing in software engineering and prior work on cross prover interoperability.

1 Introduction

Software engineers are familiar with content-addressing. Git identifies every commit, tree, and blob by the SHA-1 hash of its content [1]. The Nix package manager derives store paths from the hash of all build inputs [2]. IPFS addresses files by their cryptographic hash, making retrieval location-independent [3]. The Unison programming language names functions by the hash of their syntax tree [4].

In each case, the core idea is the same: derive the identifier from the *content itself*, not from a human-chosen name or a filesystem location. The result is an identifier that is *stable* (it does not change when the content moves), *universal* (it requires no central authority), and *self-authenticating* (anyone can verify it by rehashing).

Formal mathematics has not yet adopted this pattern. Theorems in proof assistants—Lean [5], Coq [6], Isabelle [7], Agda [8]—are identified by fully qualified names tied to project structure, file paths, and namespace conventions. Renaming a file, moving a declaration to a different module, or reorganizing a namespace hierarchy breaks every downstream reference, even though the mathematical content is unchanged. This problem is not hypothetical: it occurs routinely in Mathlib, the largest Lean 4 library, which undergoes continuous refactoring across its ∼200,000 declarations [9]. A complementary line of work addresses the downstream consequence—automated proof repair [10, 11]—but does not eliminate the root cause: names are not intrinsic to the mathematics.

This paper asks: *can we apply content-addressing to formal mathematics?* We show that the answer is yes, with important caveats about the granularity of equivalence (Section 5). We describe a working implementation for Lean 4 and a decentralized registry architecture that requires no infrastructure beyond Git.

Contributions. A content-addressing pipeline for Lean 4 declarations: canonicalization, serialization, and SHA-256 hashing (Section 4). Two canonicalization levels with different stability/e-

quivalence trade-offs (Section 4.1). A Merkle DAG hashing mode that gives declarations truly content-determined identity across project boundaries (Section 4.2). A decentralized registry architecture exploiting the monotonicity of mathematical proof (Section 7). Empirical evaluation on 57,613 theorems from a Mathlib module environment (Section 4.3). An open-source implementation as a Lean 4 library and CLI tool (Section 6). A working implementation if available at https://github.com/marcellop71/CA

2 Related Work

Content-addressing in software and programming languages. Content-addressed storage is well-established in software systems (Git [1], IPFS [3], Nix [2]). The closest analog in programming languages is Unison [4], which was designed from the ground up around content-addressing: names are metadata, refactoring never invalidates hashes, and the full syntax tree is hashed. We differ in three ways: we *retrofit* content-addressing onto an existing language not designed for it; we hash only the *type*, not the implementation (justified by proof irrelevance); and canonicalization in a dependently typed setting is complicated by definitional equality and universe polymorphism.

Formal mathematics infrastructure. Hurd's OpenTheory [19] tackled the same core problem—prover-independent theorem identity—for the HOL family, using theory interpretations rather than cryptographic hashing. Gauthier et al. [20] and Müller et al. [21] align concepts *across* type theories; our content-addressing operates *within* one, where canonical forms can be compared syntactically—a natural prerequisite for cross-system alignment. The OMDoc/MMT framework [22, 23] and the vision of Kohlhase and Rabe [24] articulate a pluralistic formal library; we provide a concrete identity mechanism for it. Existing library management (Mathlib's monorepo [9], Coq's opam packages [6], Isabelle's AFP [30]) relies on name-based identity throughout.

Proof repair and deduplication. Ringer et al. [10] and Gopinathan et al. [11] repair proofs broken by library refactoring; content-addressing aims to prevent such breakage rather than mitigate it. Type-based deduplication of theorem statements is used in AI-for-theorem-proving pipelines [27], essentially Level 0 without cryptographic persistence. The question of normal forms in dependent type theory [17, 18] underlies our canonicalization; we sidestep full normalization with pragmatic partial levels.

3 Name-based identity vs content-addressing

This section describes the problem and the proposed solution at a high level. We first show how Lean's name-based identity couples references to project structure, file layout, and naming conventions, making them fragile under refactoring and opaque across project boundaries. We then introduce content-addressing—hashing the canonicalized type expression rather than the name—and state its key properties.

Name-based Identity To use a theorem in Lean 4, a programmer writes something like:

Listing 1: Using a theorem by its fully qualified name

```
import Mathlib.Data.Finset.Sum

example : forall (s : Finset Nat) (f g : Nat -> Int),
```

```
    s.sum (f + g) = s.sum f + s.sum g :=
  Finset.sum_add_distrib
```

This single reference, `Finset.sum_add_distrib`, encodes three pieces of information beyond the mathematics: (1) the **project** (Mathlib), (2) the **file** (`Mathlib/Data/Finset/Sum.lean`, needed for the `import`), and (3) the **namespace path** (`Finset.sum_add_distrib`, a human-chosen convention). None of these are intrinsic to the mathematical statement. The theorem says that finite sums distribute over addition—a fact that is true independently of what we call it or where we store it.

When any of these coordinates change, references break. A theorem renamed for consistency (`sum_add_distrib` → `sum_add`), a declaration relocated to a different module (`Data.Finset.Sum` → `Algebra.BigOps`), a namespace hierarchy flattened or reorganized, a project forked with new prefixes—each is a no-op mathematically, but a breaking change syntactically. The cost is borne by every downstream project: CI pipelines fail, students' homework stops compiling, automated tooling must track name migrations.

Within a single library like Mathlib, naming conventions and import paths are at least centrally managed. Across independent projects, the situation is worse. There is no standardized way for project A to refer to a result proved in project B, other than adding a direct dependency on B's entire package—including its specific version, file layout, and naming scheme. If two independent projects prove the same theorem under different names, there is no mechanism to recognize that they state the same mathematical fact. Each project is an island, and the only bridges are explicit package dependencies with name-level coupling.

This problem will only grow. Mathlib is evolving into the shared foundation that virtually all Lean projects depend on, but the ecosystem is expanding well beyond pure mathematics: formalization projects in cryptography, blockchain protocol verification, physics, and other applied domains are emerging, each building on Mathlib and on each other. As the number of independent projects grows, so does the cost of name-level coupling—every cross-project reference is a fragile link that any upstream refactoring can break.

Content-Addressing The central idea is simple: *hash the mathematics, not the name.*

Given a declaration with a type expression (its mathematical statement), we: (1) **canonicalize** the type expression to remove representation artifacts, (2) **serialize** the canonical expression to a byte sequence, and (3) **hash** the byte sequence with SHA-256. The resulting 256-bit hash is the declaration's *content address*. For theorems, the content address is therefore determined entirely by the statement—the proof term is never hashed. For propositions, this is inevitable given proof irrelevance.

Content addresses have four key properties. They are *stable*: the address does not change when a declaration is renamed, moved, or reorganized. They are *universal*: no project, file, or import context is needed, and no coordination or central authority is required. They are *self-authenticating*: anyone can verify an address by recomputing the hash from the declaration's type. And they are *collision-resistant*: two mathematically different statements will (with overwhelming probability) have different addresses.

An important caveat: content addresses identify declarations that share the same *canonical type*, but this does not make them freely interchangeable. Declarations carry metadata (attributes, instance registrations) that the hash does not capture, and definitions with the same type may compute differently. We discuss these limits in Section 5.

3.1 Features

Search by statement, not by name Content-addressing inverts the usual discovery workflow. To determine whether a result is already known—proved, published as an open conjecture, or simply mentioned in a registry—one does not need to guess the name a library author might have chosen, nor search through documentation by keyword. Instead, it suffices to *write the statement* as a Lean type expression, compute its content address (a single call to the hashing pipeline), and look up that address in the registry. A match means the exact mathematical fact is already recorded, regardless of the name, file, or project it lives in. A miss means no project in the registry has published that statement at the chosen canonicalization level. The search is determined entirely by the mathematics—no text matching, no naming conventions, no knowledge of library structure required.

Address is stable under proof improvement [Stable under proof improvement] Because the content address depends only on the statement, improving a proof—making it shorter, faster, or more elegant—does not change the address. The mathematical "API" is stable even as implementations improve. A library can silently replace a slow proof with a faster one and no downstream content address is affected. It's the analogue of "you can refactor the implementation without changing the interface" in software.

Content addresses as permanent citations Academic papers about formal mathematics currently cite theorems by library name (e.g., `Finset.sum_add_distrib`). These citations break silently when libraries refactor. A content address serves as a permanent reference to a mathematical fact. A paper citing `ca:a1b2c3...` remains valid regardless of future renaming or reorganization.

Memoization for proof search (AI angle) In automated or AI-assisted proof search, the same goal type often arises in different proof contexts. A content address canonically identifies the goal independently of the surrounding proof state. This enables a global lookup table: before attempting synthesis, check whether the goal's content address already appears in the registry. If it does, the proof can be retrieved rather than re-discovered.

Version-independent references A content address for a theorem is the same whether computed from Mathlib v4.3 or v4.9, provided the mathematical statement has not changed. This decouples mathematical identity from library versioning: a project can depend on a *mathematical fact* rather than on a specific version of a specific package.

Progress tracking for formalization campaigns A formalization effort (e.g., all theorems in a textbook chapter) can be specified as a list of content addresses registered as open points. Progress is then the fraction of addresses whose status has moved from `open` to `proved`, a well-defined, cross-project metric that updates automatically via the status cascade.

Toward granular building. Lean's build system (Lake) operates at the granularity of entire modules: importing a single declaration requires compiling the full module and all its transitive dependencies. This monolithic model compounds the cost of name-based coupling—a project that needs one lemma from a large module pays for thousands of unrelated declarations. Content-addressing opens a path toward finer granularity. If declarations are individually addressable by

content, a build system could in principle fetch and verify only the specific results a project actually uses, rather than pulling in entire module trees. Granular addressing would lead naturally to granular imports: referencing individual facts rather than files. This is a harder problem in a compiled ecosystem like Lean, where elaboration, compilation, and kernel-checking are tightly coupled at the module level, but content-addressing provides the necessary first step—a stable, module-independent name for each declaration.

4 A working pipeline

The address of a declaration is computed in three stages: canonicalization, serialization, and hashing.

4.1 Canonicalization

Canonicalization transforms a Lean 4 `Expr` (expression) into a canonical form where expressions differing only in irrelevant ways become syntactically identical. We implement two levels, offering a trade-off between stability and the strength of the equivalence relation.

Level 0: structural canonicalization. Level 0 is a pure transformation (no monadic context required) that normalizes two sources of superficial variation. First, *universe parameters*: Lean uses named universe polymorphism, and these names are arbitrary; Level 0 renames them to positional indices (`u_0`, `u_1`, ...) by order of first occurrence. Second, *metadata annotations*: Lean attaches `.mdata` nodes to expressions for internal bookkeeping (source positions, elaboration hints); these carry no mathematical content and are stripped.

Lean 4 uses de Bruijn indices [14] for bound variables, so α-equivalence is handled by the representation itself. Binder *names* (e.g., `n` vs. `x`) are cosmetic and excluded from hashing. Level 0 is fast and deterministic. It depends only on the structure of the `Expr` type, which is stable within a major Lean version but has changed across major versions (e.g., Lean 3 to Lean 4). Its limitation is that it does not normalize definitional equalities: two formulations that differ by an unfolded abbreviation or a reduced beta-redex receive different hashes.

Level 1: reducible normalization. Level 1 extends Level 0 with *reducible-transparency normalization*. It requires a `MetaM` (meta-programming monad) context because it invokes Lean's `whnf` (weak head normal form) reduction. The normalizer, `deepNormalize`, recursively applies `whnf` at every subexpression under `reducible` transparency:

Listing 2: Deep normalization (simplified pseudocode)

```
deepNormalize(e, depth):
  if depth >= 256: return e       -- recursion guard
  e <- whnf e                     -- reducible transparency
  match e:
    .app f a      -> .app (deepNormalize f) (deepNormalize a)
    .lam _ t b _  -> normalize t; open binder; normalize body;
                     re-abstract
    .forallE _ t b _ -> same as lambda
    .letE _ _ v b -> normalize v; substitute; normalize result
    .mdata _ e    -> deepNormalize e
    .proj _ _ e   -> normalize e; whnf the projection
```

```
    _                -> return e
```

Under reducible transparency, notation aliases (e.g., `HAdd.hAdd` → `Add.add`), declarations marked `@[reducible]`, beta-redexes, let-bindings, and reducible structure projections are all unfolded. Crucially, `@[semireducible]` declarations (including typeclass instances) are *not* unfolded, preserving abstraction boundaries: `Add Nat` stays as `Add Nat` rather than being replaced by its instance implementation.

As an example, consider `theorem add_comm_v1 : ∀ (a b : Nat), a + b = b + a` and a variant `add_comm_v2 : ∀ (a b : Nat), b + a = a + b`. At Level 0, these may receive different hashes if the elaborated expressions differ structurally. At Level 1, after `whnf` normalization, both reduce to the same canonical form and receive the same hash. On `Init.Data.Nat.Basic`, the largest equivalence class grows from 391 members (Level 0) to 467 (Level 1)—76 additional declarations collapsed by reducible unfolding. Section 4.3 presents a larger-scale evaluation.

Level 2: full definitional equality (open problem). The ideal canonicalization would assign the same hash to all definitionally equal types. However, Lean's dependent type theory does not admit unique normal forms under full definitional equality [17]. Achieving Level 2 would require either restricting the fragment of the type theory considered, or developing new normalization techniques. We leave this as an open problem (see Section 5 for further discussion).

4.2 Serialization, Hashing, and Hashing Modes

Serialization. The canonical expression is serialized to a `ByteArray` using a tagged encoding: each `Expr` constructor is assigned a unique tag byte, followed by the serialization of its children. Binder names are omitted (de Bruijn indices suffice). `BinderInfo` (implicit, explicit, instance, etc.) is included for lambda and forall nodes. `.mdata` is transparent: the child is serialized directly. For `.const name levels`, the treatment of `name` depends on the hashing mode (below).

Hashing. The serialized byte array is hashed with SHA-256, producing a 64-character hexadecimal string. We use SHA-256 via OpenSSL's EVP API through a minimal C FFI shim, chosen for ubiquity and ease of binding rather than performance (a faster hash such as BLAKE3 would work equally well—the choice of hash function is orthogonal to the addressing design).

Hashing modes. When serializing a constant reference (`.const name levels`), there is a choice of what to emit for `name`. In *name-based* mode (default), the declaration's `Name` is serialized as a UTF-8 string—fast and stable within a single library, but if a dependency is renamed, the address changes. In *content-based* (Merkle DAG) mode, the 32-byte content hash of the referenced declaration is emitted instead. Declarations are processed in topological (dependency) order via Kahn's algorithm [16], so each hash incorporates the hashes of its transitive dependencies. The result is a Merkle DAG [15] where a declaration's identity depends only on the mathematical content of everything it (transitively) references.

Mutually recursive declarations form cycles in the dependency graph. The topological sort detects these and falls back to name-based hashing for cycle members—a conservative choice that preserves soundness at the cost of weaker identity for mutual definitions. Mutual recursion is uncommon in theorem *statements*, but Lean's elaboration of inductive types generates mutually recursive auxiliary declarations (`rec`, `casesOn`, etc.), so the fallback affects more declarations than one might initially expect when indexing definitions alongside theorems.

Mode	Constant references become	Trade-off
Name-based	The declaration's `Name` string	Fast; changes if deps renamed
Content-based	32-byte content hash of the dep	True content identity; slower

Table 1: Hashing modes.

4.3 Results

This section presents both concrete examples of content-addressing in action and a quantitative evaluation on Mathlib.

Duplicate detection. Large libraries inevitably accumulate duplicates—theorems with different names but identical mathematical content. Content-addressing detects these automatically: duplicates share the same hash. The technique of comparing type expressions for deduplication is already used in the AI-for-theorem-proving community—for instance, synthetic theorem generation pipelines deduplicate candidates by `Expr` equality [27]. Content-addressing provides a principled, persistent, and cross-project version of what is currently done ad hoc within individual pipelines.

Cross-project references. Consider two independent projects that both work with a mathematical concept:

Listing 3: Two projects proving the same lemma independently

```
-- In project-a/Algebra/Basic.lean
theorem add_comm_nat : forall a b : Nat, a + b = b + a :=
  Nat.add_comm

-- In project-b/MyMath/Nat.lean
theorem nat_addition_commutative : forall a b : Nat, a + b = b + a :=
  fun a b => Nat.add_comm a b
```

Despite different names, files, and projects, both declarations receive the same content address. A search tool can discover that these are the same mathematical fact, enabling cross-project deduplication and discovery without any coordination between the projects.

Open problems and conditional results. Content-addressing supports a structured workflow for open problems, though this aspect of the system is prospective—the mechanism is implemented but has not yet been exercised at scale. A researcher publishes a conjecture as an *open point*—a `Prop` with a content address but no proof:

Listing 4: An open conjecture and a conditional result

```
@[open_point "Bertrand's postulate, elementary form"]
def BertrandPostulate : Prop :=
  forall n : Nat, n >= 1 -> exists p, Nat.Prime p /\ n < p /\ p < 2 * n

@[publish "Consequence of Bertrand's postulate"]
theorem prime_between (n : Nat) (h : BertrandPostulate) (hn : n >= 1) :
    exists p, Nat.Prime p /\ n < p /\ p < 2 * n :=
  h n hn
```

The registry (Section 7) tracks which conditional results depend on which open points. When someone eventually proves the conjecture, the status of all downstream results cascades from "conditional" to "proved." Several practical questions remain open: how to handle conjectures that turn out to be false (or independent), how to distinguish `sorry`-based development from genuinely axiomatic assumptions, and how to curate status metadata across projects. We discuss these further in Section 6.1.

Registry generation. A project publishes its annotated declarations by adding `#ca_registry "registry/"` at the end of a Lean file that imports the annotated modules. When `lake build` compiles this file, it writes `declarations.json` (address, name, status, type, deps for each declaration) and `meta.json` (project summary with counts by status) to the specified directory.

Evaluation on Mathlib. We evaluated the pipeline on the environment loaded by `Mathlib.Algebra.Group.Bas` a core module of Mathlib that transitively imports 1,209 modules covering foundational arithmetic, data structures, and algebraic hierarchy definitions. The pipeline was run in name-based mode at Level 1 (reducible normalization), restricted to theorems. All 57,613 theorems in the transitive environment were processed successfully—no declarations were skipped or failed.

The 57,613 theorems produce 57,036 unique type addresses. Of these, 544 addresses are shared by more than one theorem, covering 1,121 theorems (1.9% of the total).

Class size	**Classes**	**Theorems**
1 (unique)	56,492	56,492
2	513	1,026
3	29	87
4	2	8
Total	57,036	57,613

Table 2: Equivalence class size distribution (L1, name-based, theorems only).

Examples of detected duplicates. The equivalence classes reveal genuine duplicates—theorems that exist under multiple names due to historical renaming, namespace aliasing, or independent re-statement:

Listing 5: Selected equivalence classes (L1)

```
Size 4:
  by_contra = by_contradiction
            = Classical.byContradiction = Classical.by_contradiction
  Type: (Not p -> False) -> p

Size 3:
  congr_arg = congrArg = let_val_congr
  Type: forall (b : a -> b), a = a' -> b a = b a'

  Nat.add_sub_cancel = Nat.add_sub_cancel_right
                     = Nat.add_sub_self_right
  Type: forall (a b : Nat), a + b - b = a
```

```
Size 2:
  mul_one = MulOneClass.mul_one
  Type: forall [MulOneClass M] (a : M), a * 1 = a

  mul_comm = CommMagma.mul_comm
  Type: forall [CommMagma G] (a b : G), a * b = b * a
```

The types shown are simplified pretty-printed representations of the canonical forms; binder names and implicit arguments are artifacts of the printer, not of the hash. These duplicates arise from distinct sources. The `by_contra` cluster contains four names for the same classical reasoning principle, accumulated as Lean's standard library evolved. The `mul_one` / `MulOneClass.mul_one` pair reflects the algebraic hierarchy: the root name and the typeclass-qualified name coexist as aliases. The `Nat.add_sub_cancel` triple shows the same arithmetic fact named by three conventions. In all cases, the content address correctly identifies these as stating the same mathematical fact, modulo Level 1 canonicalization.

Coverage. All 57,613 theorems were hashed without error, demonstrating that the pipeline (including the micro-batching workaround for `whnf`; Appendix 6.1) handles the full transitive closure of a Mathlib module. The current evaluation covers a single module's environment at Level 1 only. A systematic comparison of Level 0 vs. Level 1 across multiple Mathlib entry points, and evaluation in content-based (Merkle DAG) mode, remain as future work.

5 Equivalence Classes and Their Limits

Two declarations sharing the same content address state the same mathematical fact (modulo the canonicalization level). The set of all declarations with a given address forms an *equivalence class*.

Same hash does not mean interchangeable. The content address captures the *type*, not the proof term, runtime behavior, or elaboration metadata. This has important consequences. *Theorems in direct proofs* are interchangeable, by proof irrelevance. *Theorems in tactics* are not: a tactic like `simp [Nat.add_mul]` relies on the `@[simp]` attribute attached to that specific *name*; substituting an alias with the same hash but different attributes would change the tactic's behavior. *Definitions* are not interchangeable: two definitions with the same type may compute differently. *Instances* are not interchangeable: typeclass instances are registered by name and participate in instance resolution.

For this reason, resolution is always *by name*. The content address is a stable, project-independent key for *discovery* and *cross-referencing*, but resolution goes through a specific declaration name that carries the necessary metadata.

Listing 6: Alias discovery via content address

```
Resolved: Nat.add_mul
  Hash:   e1ea8f392f15...  (L1)
  Module: Init.Data.Nat.Basic
  Kind:   theorem
  Type:   forall (n m k : Nat), (n + m) * k = n * k + m * k
  Aliases (1): Nat.right_distrib
```

Canonicalization and definitional equality. The fundamental question underlying the equivalence relation is: which expressions should receive the same hash? The ideal answer is "all definitionally equal expressions," but this is not achievable in general. Lean's Calculus of Inductive Constructions (CIC) with proof irrelevance does not admit unique normal forms under full definitional equality [17, 18]. The reduction rules—β, δ (at various transparencies), ι, ζ, and proof irrelevance—do not confluently normalize all expressions to a unique representative.

Our two-level approach is a pragmatic compromise. Level 0 captures syntactic equality modulo universe renaming and metadata—a conservative but stable equivalence. Level 1 captures equality modulo reducible δ-reduction, β-reduction, ζ-reduction, and reducible ι-reduction—a strictly coarser equivalence that collapses common aliases. The gap between Level 1 and full definitional equality is large: semireducible and opaque definitions are not unfolded, so type synonyms at these transparencies create distinct hashes. In principle, one could add intermediate levels (semireducible transparency, instance unfolding), but each level increases computation cost, reduces stability across Lean versions, and risks divergent normalization.

Universe polymorphism. Lean uses *named* universe polymorphism: declarations are parameterized over universe level variables with programmer-chosen names. Two declarations with identical bodies but different universe parameter names (e.g., `.{u, v}` vs. `.{a, b}`) should hash identically. Level 0 handles this by renaming universe parameters to positional indices: the first universe variable encountered becomes `u_0`, the second `u_1`, and so on. The traversal order is deterministic (left-to-right depth-first over the type expression).

6 Implementation

The implementation is a Lean 4 package (`CA`) available at `https://github.com/marcellop71/CA` (repository to be made public upon acceptance). Downstream projects depend on it with a single line:

```
require ca from git "https://github.com/marcellop71/CA" @ "main"
```

End-to-end workflow. A user who wants to content-address their project performs four steps: (1) add `require ca` to their `lakefile.lean` and run `lake update`; (2) annotate declarations with `@[publish]` or `@[open_point]` (directly or retroactively via `attribute [publish] SomeName`); (3) add `#ca_registry "registry/"` to a Lean file that imports the annotated modules; (4) run `lake build`—the registry folder is generated automatically during compilation, no separate tool invocation required. The generated `registry/` folder is committed to Git and pushed; the project is now a node in the decentralized registry.

CLI. For batch processing (e.g., indexing all of Mathlib), a CLI executable provides `ca address` (compute hashes, export JSON manifest and TSV edge list) and `ca fetch` (store to Redis). Redis serves as an interactive lookup layer: given a content address, it returns all known names, types, and modules in constant time—a capability the static JSON manifest does not provide efficiently at scale.

Build and dependencies. The default `lake build` compiles the `CA` library, which requires only OpenSSL. The CLI executable (`lake build ca`) additionally requires hiredis for Redis-backed

Module	Role
`CA.Canonical`	L0 (pure) and L1 (MetaM) canonicalization
`CA.SHA256`	SHA-256 via OpenSSL EVP (C FFI)
`CA.ExprHash`	Expr serialization, batch hashing, Merkle DAG
`CA.Export`	JSON manifest, TSV edge list, statistics
`CA.Util`	Dependency collection, declaration classification
`CA.Registry.Basic`	Entry types, persistent extensions, query API
`CA.Registry.Attributes`	`@[publish]`, `@[open_point]` registration
`CA.Registry.Generate`	`#ca_registry` command, JSON output

Table 3: Module overview.

lookups. A Nix flake provides a development shell with the Lean toolchain and all native dependencies for both Linux and macOS. The implementation currently targets Lean 4.29.0-rc1 and tracks the Lean release cycle; it depends on the Batteries library [29] for `NameMapExtension`.

6.1 Limitations and Future Work

Current limitations. Level 0 and Level 1 canonicalization do not capture full definitional equality; the gap between Level 1 and the ideal is large and may be fundamentally unclosable. Level 1 addresses depend on Lean's `whnf` behavior, which can change across toolchain versions; Level 0 addresses are stable within a major Lean version, while Level 1 should be considered best-effort within a given toolchain. Lean's native `whnf` (implemented in C) lacks heartbeat checks and crashes on large environments; a micro-batching workaround is effective but inelegant (Appendix 6.1). Applying `whnf` to definition values causes process-level memory growth that crashes regardless of batch boundaries, limiting content-addressing of definitions (as opposed to theorems, where proof irrelevance makes value hashing unnecessary). Finally, Lean 4's elaboration pipeline does not support user-defined fallbacks for unresolved names; our resolution mechanism requires explicit syntax (`resolve%`), preventing seamless integration.

Future work. Several directions are worth pursuing. *Resolution elaborators*: implement `use`, `use!`, and `resolve%` as Lean 4 elaborators that query the registry, resolve by content address, and dynamically import modules. The intended syntax (not yet implemented):

Listing 7: Prospective resolution syntax (not yet implemented)

```
-- resolve% looks up a content address in the registry and
-- returns the name of a matching declaration in scope
example : forall a b : Nat, a + b = b + a :=
  resolve% "a1b2c3d4e5f6..."   -- resolves to Nat.add_comm
```

Ecosystem-wide indexing: apply content-addressing to the entirety of Mathlib to produce a global index of equivalence classes, enabling large-scale duplicate detection, library health metrics, and structured navigation. *Open-problem workflows*: develop the open-point and conditional-result mechanism (Section 4.3) into a robust workflow, addressing false conjectures, `sorry`-based vs. axiomatic assumptions, and cross-project status curation. *AI-assisted proof search*: content addresses provide a stable, type-based index for machine learning models to retrieve relevant lemmas by querying the content address of a goal type. *Cross-system alignment*: explore content-addressing

in other proof assistants (Coq, Isabelle, Agda) and investigate whether addresses can be made comparable via shared canonicalization. *Intermediate canonicalization levels*: investigate canonicalization at semireducible transparency or with selective instance unfolding. *Formal verification*: prove (in Lean itself) that the canonicalization and serialization stages preserve the intended equivalence relation.

Engineering Obstacles A practical obstacle to Level 1 canonicalization at scale: Lean's native `whnf` (implemented in C) contains no heartbeat or memory checks. When processing large environments (∼58,000 declarations in Mathlib modules), two failure modes arise. First, *MetaM state accumulation*: a single `MetaM` session crashes after ∼25,000 `whnf` calls due to cumulative growth of caches, discriminant trees, and instance tables; the crash point depends on call count, not on which specific declarations are processed. Second, *process-level memory growth*: applying `whnf` to definition values causes monotonic RSS growth that eventually crashes regardless of batch boundaries.

Micro-batching. Our workaround is to process declarations in micro-batches of 10, each in a fresh `MetaM` session. Types receive Level 1 normalization; values are always canonicalized with Level 0 only (no `whnf`). For theorems, the value hash is the sentinel `PROOF_IRRELEVANT` (proof irrelevance). The result is an asymmetric strategy: **types get L1, values get L0**. Hash computation is split into two phases (canonicalize+hash vs. pretty-print) in separate `MetaM` sessions, with a flush every 5,000 declarations to bound live `Expr` objects in memory.

An upstream fix—heartbeat checks and resource guards in Lean's native `whnf` implementation—would eliminate the need for micro-batching entirely.

7 Decentralized Registry

Content addresses give every declaration a stable, location-independent identity. The registry organizes the mutable metadata—who proved what, what is open, what depends on what—around this immutable core.

Every formalization in Lean already lives in a Git repository. The decentralized registry exploits this directly: each participating project hosts a `registry/` folder in its own repo, and the global registry is the union of all such folders. No additional infrastructure is needed—no IPFS, no blockchain, no central server.

This simplicity is possible because **mathematical consensus is trivial**. Unlike distributed systems that must agree on ordering or resolve write conflicts, a mathematical registry has a monotonic truth condition: a proposition is proved if *any* repository contains a valid proof. Proofs cannot be retracted—once proved, always proved. Two repositories proving the same proposition under different names is not a conflict; it is redundancy, and the content address shows they state the same fact. There is nothing to vote on, no quorum to reach. This observation connects to the broader vision of a pluralistic formal library where multiple communities contribute to a shared body of verified knowledge [24].

In concrete terms:

- **Publication** is `git push`—a project generates its `registry/` folder, commits it, and pushes.
- **Verification** is `lake build`—Lean's kernel type-checks the proofs; anyone can clone and verify independently.
- **Consensus** is trivial: one valid proof is enough.
- **Discovery** is a curated list of known repositories (`sources.json`).

Annotation attributes. Authors annotate their code with two attributes provided by the `CA.Registry` module. `@[open_point]` marks a `def X : Prop` as an open problem—a mathematical statement published without a proof. Applying it to a theorem or a non-`Prop` definition is a compile-time error. `@[publish]` marks any declaration (theorem, definition, axiom) for publication to the registry. Both attributes accept an optional description string and support retroactive application from a separate file, allowing annotation of existing code or upstream dependencies without modifying the original source:

Listing 8: Retroactive annotation of existing declarations

```
import CA
import MyProject.Theorems

attribute [open_point "My conjecture"] MyProject.SomeConjecture
attribute [publish "Key lemma"]        MyProject.some_lemma
```

Declaration status. Status is computed automatically from the Lean environment:

Pattern	Status	Rule
`theorem foo : P := proof`	proved	Type-checks, no `sorry`
`def P : Prop` with `@[open_point]`	open	Explicitly marked
`theorem bar (h : OpenP) : Q`	conditional	Proved modulo open hypotheses

Table 4: Automatic status classification.

For conditional declarations, the registry records which open addresses are required—the declarations whose proof would promote the entry from conditional to proved.

Registry folder. The `registry/` folder is generated locally inside each project. The `#ca_registry "registry/"` command (or the `ca registry` CLI) loads the compiled environment, collects annotated declarations, computes content addresses, classifies status, and writes two files:

- `declarations.json`—the append-only manifest. Each entry contains the content address, name, module, kind, status, pretty-printed type, and type dependencies.
- `meta.json`—project-level metadata (hash level, open/proved/conditional counts).

Entries are never removed. Status updates (open → proved) are reflected by updating the status field in place; Git history provides the audit trail.

Status cascade. When a proof is submitted for an open point, conditional declarations downstream may become unconditionally proved:

1. A previously open address acquires a proof—status becomes `proved`.
2. All `conditional` entries whose open dependencies included the newly proved address are re-evaluated.
3. If all of an entry's open dependencies are now proved, it is promoted to `proved`.
4. This cascades transitively.

Cross-project cascades emerge when a consumer merges multiple registries: an open point in project A may be proved in project B, promoting conditional entries in project C, with no coordination between A, B, and C. The dependency graph also makes *impact* computable: the impact of an open point is the number of conditional declarations transitively downstream of it.

Sources and discovery. Because the registry is fully decentralized, a discovery mechanism is needed so that resolution tools know where to look. A curated `sources.json` file in the CA repository enumerates known Git repositories that host a `registry/` folder:

Listing 9: `sources.json` format

```
{ "registries": [
    { "url": "https://github.com/leanprover-community/mathlib4",
      "registry_path": "registry/" },
    { "url": "https://github.com/user/my-project",
      "registry_path": "registry/" }
  ]
}
```

Adding a project to the ecosystem requires five steps: (1) add `require ca` to the project's `lakefile.lean`; (2) annotate declarations with `@[publish]` or `@[open_point]`; (3) add `#ca_registry "registry/"` to a Lean file that imports the annotated modules; (4) `lake build`, commit `registry/`, and push; (5) open a PR adding one entry to `sources.json`. Step 5 is the only coordination point; everything else is local to the project. The static list is the right starting point—it can later be replaced by automatic discovery (e.g., a GitHub topic tag or a DHT) without changing the rest of the architecture.

Current maturity. The architecture described here is implemented but early-stage. Discovery relies on a manually curated `sources.json`, there is no mechanism for revoking or correcting erroneous entries, and metadata conflicts (e.g., two projects assigning different status to the same address) are resolved by the consumer at merge time. The theoretical elegance of monotonic consensus is real, but the tooling around it is minimal.

8 Conclusion

We have presented a content-addressing system for formal mathematics, implemented as a Lean 4 library. By hashing canonicalized type expressions, we assign declarations stable identifiers determined by their mathematical content rather than their human-chosen names.

Content-addressing does not require full normalization to be useful. Even partial canonicalization (Level 0) provides meaningful stability and cross-project discoverability. The decentralized registry architecture, made possible by the monotonicity of mathematical proof, requires no infrastructure beyond Git—though significant engineering work remains to make it practical at ecosystem scale.

The system is a working prototype, and important challenges remain—most fundamentally, the gap between our canonicalization levels and full definitional equality. We hope this work demonstrates that content-addressing is a productive lens through which to view the organization of formal mathematical knowledge, and that the tools presented here are a useful step toward a more interconnected ecosystem for formalized mathematics.

References

[1] L. Torvalds and J. Hamano, "Git: Fast version control system," `https://git-scm.com`, 2005.

[2] E. Dolstra, M. de Jonge, and E. Visser, "Nix: A safe and policy-free system for software deployment," in *Proc. LISA*, USENIX, 2004, pp. 79–92.

[3] J. Benet, "IPFS—Content addressed, versioned, P2P file system," *arXiv preprint arXiv:1407.3561*, 2014.

[4] P. Chiusano and R. Bjarnason, "Unison: A friendly programming language from the future," `https://www.unison-lang.org`, 2019.

[5] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Proc. CADE-28*, Springer LNCS, vol. 12699, 2021, pp. 625–635.

[6] The Coq Development Team, "The Coq proof assistant," `https://coq.inria.fr`, 2024.

[7] T. Nipkow, L. C. Paulson, and M. Wenzel, *Isabelle/HOL—A Proof Assistant for Higher-Order Logic*, Springer LNCS, vol. 2283, 2002.

[8] U. Norell, "Towards a practical programming language based on dependent type theory," Ph.D. thesis, Chalmers University of Technology, 2007.

[9] The Mathlib Community, "Mathlib4: The math library of Lean 4," `https://github.com/leanprover-community/mathlib4`, 2024.

[10] T. Ringer, R. Palmkvist, I. Sergey, M. Gligoric, and Z. Tatlock, "Proof repair across type equivalences," in *Proc. PLDI*, ACM, 2021, pp. 112–127.

[11] K. Gopinathan, M. Sergey, and I. Sergey, "Mostly automated proof repair for verified libraries," in *Proc. PLDI*, ACM, 2023.

[12] T. Coquand and C. Paulin-Mohring, "Inductively defined types," in *COLOG-88*, Springer LNCS, vol. 417, 1990, pp. 50–66.

[13] M. Carneiro, "Lean's type theory," `https://github.com/digama0/lean-type-theory`, 2019.

[14] N. G. de Bruijn, "Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem," *Indagationes Mathematicae*, vol. 34, pp. 381–392, 1972.

[15] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Advances in Cryptology—CRYPTO '87*, Springer LNCS, vol. 293, 1988, pp. 369–378.

[16] A. B. Kahn, "Topological sorting of large networks," *Communications of the ACM*, vol. 5, no. 11, pp. 558–562, 1962.

[17] A. Abel, J. Öhman, and A. Vezzosi, "Decidability of conversion for type theory in type theory," *Proc. ACM Program. Lang.*, vol. 2, no. POPL, 2018, pp. 23:1–23:29.

[18] T. Coquand, "An algorithm for testing conversion in type theory," in *Logical Frameworks*, Cambridge Univ. Press, 1991, pp. 255–279.

[19] J. Hurd, "The OpenTheory standard theory library," in *Proc. NFM*, Springer LNCS, vol. 6617, 2011, pp. 177–191.

[20] T. Gauthier, C. Kaliszyk, and J. Urban, "Aligning concepts across proof assistant libraries," *J. Symbolic Comput.*, vol. 90, pp. 89–103, 2019.

[21] D. Müller, C. Gauthier, M. Kohlhase, and F. Rabe, "Alignment-based translations across formal systems," 2017, manuscript.

[22] F. Rabe and M. Kohlhase, "A scalable module system," *Information and Computation*, vol. 230, pp. 1–54, 2013.

[23] M. Kohlhase, *OMDoc—An Open Markup Format for Mathematical Documents (Version 1.2)*, Springer LNAI, vol. 4180, 2006.

[24] M. Kohlhase and F. Rabe, "QED reloaded: Towards a pluralistic formal library of mathematical knowledge," *J. Formalized Reasoning*, vol. 9, no. 1, pp. 201–234, 2016.

[25] The OpenMath Society, "OpenMath: An extensible standard for representing the semantics of mathematical objects," `https://openmath.org`, 2004.

[26] A. Assaf, G. Burel, R. Cauderlier, D. Delahaye, G. Dowek, C. Dubois, F. Gilbert, P. Halmagrand, O. Hermant, and R. Saillard, "Dedukti: A logical framework based on the $\lambda\Pi$-calculus modulo theory," manuscript, 2016.

[27] S. Agarwal, A. Welleck, et al., "Synthetic theorem generation in Lean," *OpenReview preprint*, 2024.

[28] A. Abel and G. Scherer, "On irrelevance and algorithmic equality in predicative type theory," *Logical Methods in Computer Science*, vol. 8, no. 1, 2012.

[29] The Lean Community, "Batteries: Community-maintained libraries for Lean 4," `https://github.com/leanprover-community/batteries`, 2024.

[30] G. Klein, T. Nipkow, and L. Paulson, eds., "Archive of formal proofs," `https://www.isa-afp.org`, 2004.